

DEVOPS MASTER COURSE

Volume 9A

Terraform Foundations

Chapters 1-10

Infrastructure as Code, HCL, providers, lifecycle, state, iteration, modules, refactoring, team workflow, and foundation capstone

2026 Edition

DECLARE Describe desired infrastructure	PLAN Review proposed change	APPLY Execute with evidence
---	---------------------------------------	---------------------------------------

About Volume 9

Volume 9 teaches infrastructure automation as an engineering discipline rather than a collection of commands. Parts A and B build Terraform mastery; Parts C and D develop Ansible automation; Part E combines both tools with CI/CD, security, governance, testing, and operating controls; Part F completes the volume with enterprise projects, advanced troubleshooting, certifications, and interviews.

Part A is intentionally vendor-neutral. The labs use local, random, and file-based examples so learners can practice the Terraform lifecycle without cloud charges. The same mental models later transfer to AWS, Azure, Google Cloud, Kubernetes, SaaS APIs, and on-premises platforms through providers.

Core principle: Terraform is a stateful change-management system. Reliable automation requires controlled configuration, reviewed plans, protected state, pinned dependencies, explicit ownership, and reversible operating procedures.

Volume 9 Structure

Part	Primary Coverage	Outcome
9A	Terraform foundations, HCL, providers, lifecycle, state, modules, imports, team workflow	Build and operate a safe vendor-neutral Terraform project.
9B	Advanced Terraform architecture, testing, policy, multi-environment design, HCP Terraform and enterprise workflows	Design reusable and governed Terraform platforms.
9C	Ansible foundations, inventory, ad hoc operations, playbooks, variables, templates, handlers	Automate repeatable configuration safely.
9D	Roles, collections, secrets, network automation, Windows, AWX and Automation Platform	Operate Ansible at enterprise scale.
9E	Terraform plus Ansible pipelines, GitOps, security, compliance, observability and governance	Create integrated infrastructure automation systems.
9F	Enterprise projects, incidents, troubleshooting, certifications, interviews and capstone	Demonstrate production-ready automation mastery.

Safety and Lab Controls

- Use an isolated learning directory and source-control repository. Never experiment against employer or production infrastructure without written authorization.
- Do not commit credentials, state files, plan files, private keys, environment files, or generated secrets. Treat state and saved plans as sensitive artifacts.
- Use least-privilege, short-lived credentials when cloud providers are introduced. Prefer federation or managed identity over static access keys.

- Review every plan for create, update, replace, and destroy actions. A successful command is not proof that the change is safe.
- Keep backups and recovery procedures for remote state. Use locking when supported, and never force-unlock until the active writer is proven absent.
- Destroy lab resources and verify cleanup. Record exceptions when a resource must remain for a later exercise.

Learning Method

- Explain the desired outcome and risk before writing HCL.
- Use terraform fmt, validate, and plan before apply.
- Save evidence: configuration, plan summary, apply output, state inspection, validation result, and cleanup result.
- Introduce one change at a time when troubleshooting.
- Convert completed labs into a sanitized portfolio repository with a README, diagram, decisions, tests, and runbook.

Chapter 1: Infrastructure as Code and the Terraform Operating Model

Terraform is most valuable when teams need repeatable, reviewable infrastructure changes across many systems. The challenge is not merely creating resources; it is controlling ownership, change sequencing, state, drift, approvals, and recovery.

Learning Objectives

- Distinguish declarative infrastructure as code from imperative scripting.
- Explain Terraform core, providers, configuration, state, and remote APIs.
- Describe the init-plan-apply cycle and its operational gates.
- Identify appropriate and inappropriate Terraform use cases.

Core Concepts

Declarative intent: Terraform configuration describes the desired managed objects and relationships. Terraform compares configuration, prior state, and provider observations to propose actions.

Execution graph: References create dependency edges. Terraform can operate independent resources concurrently while preserving dependencies.

Providers: Provider plugins translate Terraform resource operations into remote API calls. Provider behavior, authentication, and versioning are separate concerns from Terraform core.

State: State binds resource addresses to real objects and records metadata required for future changes. It is operational data, not a casual cache.

Change control: A plan is a proposal, not an approval. Production workflows add code review, policy, security checks, human authorization, and post-change validation.

Boundaries: Terraform is strong for provisioning and lifecycle management. Configuration management, application deployment, and one-time scripts may be better handled by Ansible or purpose-built tools.

Concept	Question	Production Concern
Configuration	What should exist?	Readability, review, ownership, version control
Plan	What would change?	Replacement, deletion, unknown values, policy
Apply	Who may execute?	Identity, approval, concurrency, rollback
State	What does Terraform manage?	Confidentiality, integrity, locking, recovery
Provider	How is the remote API controlled?	Version, credentials, rate limits, behavior

Reference Configuration or Command Pattern

```
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    local = {
      source = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

resource "local_file" "welcome" {
  filename = "${path.module}/generated/welcome.txt"
  content = "Infrastructure as Code starts with controlled change."
}
```

Production Checklist

- Every managed resource has an owner and lifecycle policy.
- Configuration and dependency lock files are version controlled.
- Plans are reviewed before apply in controlled environments.
- State has a defined storage, access, backup, and recovery model.
- Generated resources are validated after apply and removed during cleanup.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Terraform proposes unexpected deletion	Resource removed or address changed	Stop; inspect configuration, state, and recent refactor	Plan no longer contains unintended destroy
Resource created outside Terraform	Unmanaged infrastructure or partial adoption	Decide whether to import, replace, or leave unmanaged	Ownership decision documented
Apply succeeds but service fails	Provisioning is not functional validation	Run service-level tests and inspect dependencies	User journey or acceptance test passes
Team members produce different plans	Different code, variables, providers, or state	Compare commit, lock file, workspace, variables, and backend	Reproduced plan inputs match

Hands-on Lab 1: Create Your First Controlled Terraform Project

Prerequisites

- Terraform CLI installed.
- A code editor and terminal.
- An empty working directory not synchronized by a consumer file-sharing tool.

Implementation

1. Create main.tf with the local provider and local_file resource shown above.
2. Run terraform fmt -check, then terraform init.

3. Run terraform validate and record the result.
4. Run terraform plan -out=tfplan and review all proposed actions.
5. Apply the saved plan with terraform apply tfplan.
6. Inspect the generated file, then run terraform destroy and verify removal.

Validation Evidence

- terraform version returns successfully.
- terraform validate reports a valid configuration.
- The saved plan contains one create action and no unexpected actions.
- The generated file content matches configuration, and destroy removes it.

Cleanup

- Delete tfplan because saved plans can contain sensitive values.
- Remove .terraform only after completing the lab; retain .terraform.lock.hcl in source control.
- Confirm no generated file remains.

Interview Questions and Model Answers

1. Why is Terraform described as declarative?

The configuration states the desired managed infrastructure. Terraform calculates the actions needed to reconcile that desired state with known and observed infrastructure.

2. What is the role of a provider?

A provider implements resource and data-source behavior for a target API and handles schema, create, read, update, and delete operations.

3. Is a successful apply sufficient evidence of success?

No. Apply proves that provider operations completed. Functional, security, reliability, and cost acceptance require separate validation.

4. When should Terraform not be the primary tool?

Avoid forcing it into high-frequency application deployment, procedural server configuration, or tasks whose remote objects have no stable lifecycle or provider support.

Chapter Summary

- Terraform manages infrastructure through configuration, providers, state, and a dependency graph.
- Plan review and post-apply validation are core controls.
- State and dependency versions must be treated as production assets.
- Choose Terraform for resource lifecycle management, not every automation task.

Chapter 2: Terraform Installation, CLI Workflow, and Project Hygiene

A reproducible Terraform workflow begins before the first resource. Engineers must control the binary, working directory, provider installation, lock file, environment variables, credentials, logs, and generated artifacts.

Learning Objectives

- Install and verify Terraform safely on Windows, Linux, or macOS.
- Explain the .terraform directory and dependency lock file.
- Use fmt, init, validate, plan, show, apply, and destroy correctly.
- Design a repository layout and ignore rules that protect sensitive artifacts.

Core Concepts

Binary provenance: Obtain Terraform through an approved package manager or official distribution channel. Record version and verify organizational requirements before upgrades.

Initialization: terraform init configures the backend, downloads child modules, installs providers, and updates dependency selections.

Dependency lock: The .terraform.lock.hcl file records provider selections and checksums. Commit it so future initialization uses predictable provider packages.

Working data: The .terraform directory is local working data and should not be committed. It may include provider binaries, modules, backend metadata, and active-workspace information.

Command discipline: fmt and validate improve consistency; plan evaluates a run context; apply executes a plan; show makes plan or state data inspectable.

Sensitive artifacts: State, crash logs, variable files, credentials, and saved plans can reveal secrets or infrastructure details and require explicit handling.

Artifact	Commit?	Reason
*.tf	Yes	Source configuration
.terraform.lock.hcl	Yes	Provider selections and checksums
.terraform/	No	Local working data and backend metadata
terraform.tfstate*	No	Sensitive state and backup snapshots
*.tfplan	No	Opaque saved plan may include sensitive values
terraform.tfvars	Usually no	May contain environment-specific secrets
README and runbook	Yes	Operating guidance and ownership

Reference Configuration or Command Pattern

```
# .gitignore
.terraform/
*.tfstate
*.tfstate.*
*.tfplan
crash.log
crash.*.log
*.auto.tfvars
*.auto.tfvars.json
.terraformrc
terraform.rc

# Recommended local workflow
terraform fmt -check -recursive
terraform init
terraform validate
terraform plan -out=tfplan
terraform show tfplan
terraform apply tfplan
```

Production Checklist

- Terraform version requirements are declared in configuration.
- The dependency lock file is committed and reviewed.
- The .gitignore excludes state, plans, logs, credentials, and local working data.
- Backend credentials are supplied through approved environment or identity mechanisms.
- CI and local workflows run the same formatting and validation checks.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
terraform command not found	Binary not installed or PATH not updated	Verify installation path and reopen terminal	terraform version succeeds
init repeatedly downloads providers	Cache absent or working directory recreated	Use approved plugin cache where appropriate; retain lock file	Second init selects expected versions
validate fails after clone	Initialization not completed or module missing	Run terraform init -backend=false for static validation	validate passes without contacting backend
Git shows state or plan files	Ignore rules added too late	Remove from index, rotate exposed secrets, purge history if required	Repository scan shows no sensitive artifacts

Hands-on Lab 2: Build a Reproducible Terraform Repository

Prerequisites

- Git installed.
- Terraform CLI available.
- A new local repository.

Implementation

1. Initialize a Git repository with main.tf, versions.tf, variables.tf, outputs.tf, README.md, and .gitignore.
2. Declare required Terraform and provider versions.
3. Run terraform fmt -recursive and terraform init.
4. Inspect .terraform.lock.hcl and confirm it is not ignored.
5. Run terraform validate and create a saved plan.
6. Use git status to prove that .terraform, state, and plan files are excluded.

Validation Evidence

- Repository contains configuration, README, .gitignore, and lock file.
- terraform init and validate succeed.
- git status does not show .terraform, tfstate, or tfplan.
- README documents prerequisites, workflow, validation, and cleanup.

Cleanup

- Delete the saved plan.
- Run terraform destroy if any resource was created.
- Keep the clean repository as the starting point for later labs.

Interview Questions and Model Answers

1. Why commit .terraform.lock.hcl?

It records provider selections and checksums so repeated initialization can use consistent provider packages and reviewers can see dependency changes.

2. Why not commit .terraform?

It is environment-specific working data that can contain binaries, modules, and backend metadata. It should be recreated with terraform init.

3. Does terraform validate replace terraform plan?

No. Validate checks configuration consistency without evaluating a complete target run. Plan includes validation and evaluates provider, state, and input context.

4. Why delete saved plan files?

Saved plans are opaque execution artifacts that can embed configuration and sensitive values. Retain them only within an approved protected workflow.

Chapter Summary

- Reproducibility depends on binary, dependency, repository, and artifact hygiene.
- Initialization creates local data that must remain outside version control.
- The lock file is a controlled dependency artifact.
- Formatting, validation, planning, and evidence should be consistent across local and CI workflows.

Chapter 3: Terraform Language: Blocks, Expressions, Variables, Locals, and Outputs

HCL is concise, but small language mistakes can create broad infrastructure impact. Engineers need a precise model for types, null and unknown values, variable validation, sensitive markings, local values, expressions, and output contracts.

Learning Objectives

- Read and write Terraform blocks and arguments.
- Use primitive and complex types deliberately.
- Apply variable validation, locals, conditional expressions, and functions.
- Design outputs as stable contracts without exposing secrets.

Core Concepts

Blocks and arguments: Blocks define constructs such as terraform, provider, resource, data, variable, locals, output, and module. Labels identify block instances; arguments assign expressions.

Types: Terraform values include string, number, bool, list, set, map, tuple, and object. Explicit variable types prevent surprising conversion and improve module contracts.

Null and unknown: null represents omission or absence. Unknown values are placeholders known only after planning or applying; design expressions to tolerate them.

Variables: Input variables should define type, description, validation, and sensible defaults only when a default is genuinely safe.

Locals: Local values name repeated expressions and normalize input, but should not hide complex business logic.

Outputs: Outputs expose useful root-module or child-module values. sensitive=true hides routine CLI display but does not remove the value from state.

Construct	Good Use	Risk
variable	Stable input contract with validation	Weak types and unsafe defaults
locals	Normalization and naming repeated expressions	Hidden policy or excessive indirection
output	IDs, endpoints, and handoff data	Exposing secrets or unstable internal details
for expression	Transform collections predictably	Changing keys and causing replacements
conditional	Select between compatible values	Deep nested logic that obscures intent

Reference Configuration or Command Pattern

```
variable "environment" {
  type    = string
  description = "Deployment environment."

  validation {
    condition = contains(["dev", "test", "prod"], var.environment)
    error_message = "environment must be dev, test, or prod."
  }
}

variable "owners" {
  type = set(string)
}

locals {
  normalized_environment = lower(trimspaces(var.environment))
  metadata = {
    environment = local.normalized_environment
    owners      = join(",", sort(tolists(var.owners)))
    managed_by  = "terraform"
  }
}

output "metadata" {
  value = local.metadata
}
```

Production Checklist

- Variables use explicit types and descriptions.
- Production inputs have validation rules and no unsafe defaults.
- Locals improve readability rather than hide policy.
- Sensitive outputs are minimized and consumers use approved secret stores.
- Collection keys are stable because key changes can alter resource addresses.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Invalid value accepted until provider error	Missing variable validation	Add domain validation near the input contract	terraform validate or plan fails with clear message
Conditional reports inconsistent types	True and false results differ	Return compatible types or normalize explicitly	Expression evaluates with intended type
Sensitive value appears in state	Sensitive flag only suppresses display	Avoid placing secret in Terraform when possible; protect state	State access reviewed and secret rotated if exposed
for_each keys change unexpectedly	Keys derived from mutable display values	Use stable identifiers as keys	Plan shows updates rather than destroy/create

Hands-on Lab 3: Model a Validated Environment Contract

Prerequisites

- Repository from Chapter 2.
- No cloud account required.

Implementation

1. Create variables for environment, owners, application_name, replica_count, and optional description.
2. Add explicit types, descriptions, and validation for environment and replica_count.
3. Create locals that normalize naming and build a metadata object.
4. Create outputs for normalized name and metadata.
5. Run plans with valid and invalid values using -var arguments.
6. Use terraform console to test lower, trimspace, merge, sort, toset, and for expressions.

Validation Evidence

- Invalid environment and replica count are rejected with useful messages.
- The normalized name is deterministic.
- Owner ordering does not create needless differences when a set is used.
- Outputs contain no credential or private-key material.

Cleanup

- Delete saved plans.
- Retain configuration for use in the module lab.
- Record expression examples in README without real secrets.

Interview Questions and Model Answers

1. What is the difference between null and unknown?

null explicitly represents absence or omission. Unknown means Terraform cannot determine the value until a later phase, often because it depends on a resource result.

2. Does sensitive=true encrypt an output?

No. It suppresses normal display in selected interfaces, but the value remains in state and may appear through authorized inspection.

3. Why prefer explicit variable types?

They create a clear contract, catch errors early, improve documentation, and reduce implicit conversion surprises.

4. When are locals harmful?

When they hide policy decisions, create long dependency chains, or make reviewers trace many indirections to understand a resource.

Chapter Summary

- HCL expressions are strongly typed and evaluated across plan and apply phases.
- Validation belongs close to input contracts.
- Stable collection keys reduce replacement risk.
- Outputs are interfaces and must be designed for security and compatibility.

Chapter 4: Providers, Resources, Data Sources, and Dependency Graphs

Providers connect Terraform to external APIs. Reliable projects must control provider source addresses, versions, authentication, aliases, resource semantics, data-source timing, and dependency edges.

Learning Objectives

- Declare required providers and provider configurations.
- Differentiate resources from data sources.
- Explain implicit and explicit dependencies.
- Use provider aliases and authentication safely.

Core Concepts

Requirements versus configuration: `required_providers` declares source and version constraints. provider blocks configure credentials, regions, endpoints, or aliases for a root module.

Resources: A resource block declares an object Terraform will create or otherwise manage. Its address becomes part of state and must remain stable through refactoring.

Data sources: A data block reads existing information. It does not make the external object Terraform-managed, and broad or ambiguous queries can make plans unstable.

Implicit dependency: A reference such as `local_file.summary.content = random_pet.name.id` creates a graph edge automatically.

Explicit dependency: `depends_on` is reserved for hidden operational dependencies that cannot be expressed through data references. Overuse reduces concurrency and can produce excessive unknown values.

Authentication: Provider credentials should come from short-lived identity mechanisms or approved environment configuration, never hardcoded HCL.

Element	Purpose	Review Question
<code>required_providers</code>	Source and compatible version	Is source trusted and constraint intentional?
provider block	Runtime configuration and alias	How are credentials supplied and scoped?
resource	Managed lifecycle object	What replacement or deletion risk exists?
data source	Read existing information	Is lookup unique, stable, and authorized?
<code>depends_on</code>	Hidden dependency edge	Why cannot a direct reference express it?

Reference Configuration or Command Pattern

```

terraform {
  required_providers {
    random = {
      source = "hashicorp/random"
      version = "~> 3.6"
    }
    local = {
      source = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

resource "random_pet" "application" {
  length = 2
  separator = "-"
}

resource "local_file" "inventory" {
  filename = "${path.module}/generated/inventory.txt"
  content = "application=${random_pet.application.id}"
}

```

Production Checklist

- Provider source and version constraints are explicit.
- The lock file change is reviewed when provider selections change.
- Credentials are external to configuration and state where possible.
- Data-source filters uniquely identify the intended object.
- Dependencies are expressed through references unless a hidden dependency is documented.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Provider unavailable during init	Registry, proxy, mirror, DNS, or constraint problem	Inspect error, CLI config, network path, source address, and lock file	init selects expected signed package
Authentication works locally but fails in CI	Different identity source or environment	Use workload identity/federation and document required permissions	CI plan succeeds without static key
Data source returns multiple matches	Lookup filter not unique	Use stable ID or stronger filter	Plan selects exactly one object
Excessive unknown values	Broad depends_on or apply-time lookup	Replace with direct references and smaller dependencies	Plan becomes more precise

Hands-on Lab 4: Create a Provider-Driven Dependency Graph

Prerequisites

- Terraform repository.
- Internet access for provider installation.

Implementation

1. Declare local and random providers with compatible version constraints.
2. Create `random_pet.application` and `local_file.inventory` with an implicit reference.
3. Run `terraform init` and inspect the lock file provider entries.
4. Run `terraform graph` and save DOT output for optional visualization.
5. Apply and verify that the generated inventory contains the random name.
6. Change a non-replacement argument, review the plan, and apply.

Validation Evidence

- The graph contains an edge from `local_file.inventory` to `random_pet.application`.
- The lock file records the provider selections.
- A second plan after apply reports no changes.
- No provider credential appears in configuration or repository.

Cleanup

- Run `terraform destroy`.
- Delete generated files and saved plans.
- Keep the graph output only if it contains no sensitive infrastructure details.

Interview Questions and Model Answers

1. What is the difference between `required_providers` and a provider block?

The requirement identifies provider source and compatible versions. The provider block supplies runtime configuration such as region, endpoint, credentials, or alias.

2. Does a data source manage the object it reads?

No. It reads information for use in configuration. Terraform does not own that object lifecycle unless it is separately represented by a resource and imported.

3. When is `depends_on` justified?

When a real operational dependency exists but no value reference can express it, such as a policy that must exist before another API action.

4. Why use provider aliases?

Aliases allow multiple configurations of the same provider, such as regions, subscriptions, projects, or accounts, while making the selected configuration explicit.

Chapter Summary

- Providers are separately versioned plugins and require deliberate source, version, and identity controls.
- Resources are managed objects; data sources are reads.
- References create the preferred dependency edges.
- Provider authentication should use short-lived least-privilege identities.

Chapter 5: Planning, Applying, Drift, Refresh, and Change Safety

The execution plan is Terraform's primary change-review artifact. Engineers must interpret action symbols, replacements, unknown values, drift, refresh behavior, saved plans, targeting, and failure recovery before allowing infrastructure mutation.

Learning Objectives

- Interpret plan actions and replacement triggers.
- Use saved plans safely in reviewed workflows.
- Explain drift detection and refresh-only operations.
- Recognize dangerous shortcuts such as routine targeting or disabled locking.

Core Concepts

Plan inputs: A plan depends on configuration, input values, provider selections, credentials, state, backend, workspace, and current remote observations.

Actions: Plans may create, update in place, replace, destroy, read, or perform no action. Replacement deserves special scrutiny because it can cause outage or data loss.

Saved plans: -out creates an opaque plan that can later be applied. Apply the exact reviewed plan in a protected workflow and discard it afterward.

Drift: Out-of-band changes may be detected during planning. Decide whether code should accept, reverse, ignore, or import the change.

Refresh-only: A refresh-only plan updates Terraform state and root outputs to match remote objects without changing remote infrastructure. Review it as carefully as other state changes.

Targeting: -target is an exceptional recovery tool, not a normal deployment strategy; it can produce incomplete changes and misleading confidence.

Plan Signal	Meaning	Required Review
+	Create	Ownership, quota, cost, naming, security
~	Update in place	Service impact and compatibility
-/+	Replace	Data migration, downtime, dependency and rollback
-	Destroy	Business owner, retention, backup, recovery
(known after apply)	Unknown value	Whether downstream decisions depend on it

Reference Configuration or Command Pattern

```
terraform fmt -check -recursive
terraform validate
terraform plan -out=tfplan
terraform show -no-color tfplan
terraform apply tfplan

# Drift investigation
terraform plan -refresh-only

# Machine-readable review artifact
terraform show -json tfplan > tfplan.json
```

Production Checklist

- The plan is generated from the same commit and inputs later applied.
- Create, update, replace, and destroy counts are summarized for reviewers.
- Destructive or replacement actions require backup and business approval.
- Targeting and -lock=false require documented emergency justification.
- Post-apply validation checks the service outcome, not only Terraform exit status.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Plan changes every run	Nondeterministic input, unstable data lookup, timestamp, or provider normalization	Compare JSON plans and remove unstable inputs	Second plan after apply is empty
Apply differs from reviewed plan	Automatic plan mode or changed inputs/state	Apply the saved approved plan; serialize runs	Applied plan checksum and run inputs match
Resource changed outside Terraform	Manual drift or external controller	Identify owner and decide reconcile, accept, ignore, or split ownership	Follow-up plan reflects intentional ownership
Partial apply failed	Provider or dependency error after some actions	Do not rerun blindly; inspect state and remote system, then re-plan	New plan explains remaining actions

Hands-on Lab 5: Create and Review a Deterministic Saved Plan

Prerequisites

- Completed provider lab.
- A clean working directory.

Implementation

1. Apply the baseline configuration and confirm a no-change plan.
2. Change local_file content and create a saved plan.
3. Inspect the human-readable plan and export JSON.
4. Record action counts and identify unknown values.

5. Apply the exact saved plan.
6. Modify the generated file manually, run a normal plan and a refresh-only plan, and explain the difference before restoring desired state.

Validation Evidence

- A baseline plan is empty.
- The saved plan contains only the intended update.
- The applied change matches the reviewed plan.
- Drift is detected and the chosen reconciliation action is documented.

Cleanup

- Restore the desired file content through Terraform, not manual editing.
- Delete saved plan and JSON plan artifacts.
- Destroy resources and verify cleanup.

Interview Questions and Model Answers

1. Why use terraform plan -out?

It creates a specific execution plan that can be reviewed and later applied without silently generating a different plan at approval time.

2. What causes replacement?

Provider schema rules, immutable remote attributes, lifecycle configuration, address changes, taint/replacement requests, or dependency changes can require destroy-and-create.

3. What is refresh-only mode?

It proposes updating state and outputs to match current remote objects without changing those remote objects.

4. Why is routine -target use risky?

It asks Terraform to focus on part of the graph, so related changes may be omitted and the resulting infrastructure may not represent the full configuration intent.

Chapter Summary

- Plans are contextual, reviewable change proposals.
- Saved plans strengthen separation between review and execution.
- Drift is an ownership and governance problem, not merely a technical difference.
- Partial failures require evidence-driven re-planning rather than blind retries.

Chapter 6: Terraform State, Backends, Locking, and Security

State is Terraform's record of resource bindings and metadata. Losing, corrupting, exposing, or concurrently modifying state can cause outages, duplicate resources, leaked secrets, or loss of management control.

Learning Objectives

- Explain what state contains and why Terraform needs it.
- Compare local and remote state approaches.
- Describe backend configuration, migration, locking, backups, and recovery.
- Operate state commands with appropriate safeguards.

Core Concepts

Binding database: State maps each resource instance address to a real remote object identifier. Terraform relies on that binding for updates and deletions.

Sensitive content: State may include resource attributes, connection information, outputs, and values marked sensitive. Access should be narrower than ordinary source-code access.

Backend: A backend defines state storage and, for some backends, locking. Backend configuration is separate from provider configuration.

Locking: When supported, Terraform automatically locks state for operations that can write it. A failed lock should stop the run rather than allow concurrent writers.

Recovery: Backups, versioning, audit logs, tested restore procedures, and clear ownership are required. A state backup that has never been restored is unproven.

State surgery: state mv, state rm, import, and force-unlock can be necessary but are high-risk operations requiring backups, exact addresses, and peer review.

Control	Purpose	Failure if Missing
Remote storage	Shared authoritative state	Conflicting local copies
Locking	Prevent concurrent writers	State corruption or conflicting changes
Encryption and access	Protect sensitive attributes	Credential or topology exposure
Versioning/backup	Recover prior snapshots	Unrecoverable state loss
Audit logging	Trace reads and writes	Weak incident evidence
Run serialization	Coordinate plan/apply	Stale plan or concurrent mutation

Reference Configuration or Command Pattern

```
terraform {
  backend "local" {
    path = "state/terraform.tfstate"
  }
}

# Inspect without directly editing the JSON file
terraform state list
terraform state show local_file.inventory
terraform state pull > protected-backup.tfstate

# Reinitialize after backend changes
terraform init -migrate-state
```

Production Checklist

- Production state is remote, encrypted, access-controlled, backed up, and audited.
- The backend supports locking or the workflow provides an equivalent single-writer control.
- Backend credentials are not hardcoded in configuration or saved plan files.
- State commands require backup, review, and exact resource addresses.
- Restore and force-unlock procedures are documented and tested in nonproduction.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
State lock cannot be acquired	Active run, abandoned lock, or backend outage	Identify lock holder and backend health; wait or recover safely	Only one writer remains and lock clears normally
Terraform wants to recreate existing object	Binding missing or wrong state selected	Stop; verify backend/workspace, then import or restore	Plan maps object without duplicate creation
State contains exposed secret	Provider stored secret attribute or output	Restrict access, rotate secret, assess logs/backups, redesign secret flow	New secret absent where feasible and access reviewed
Backend migration interrupted	Network or credential failure	Preserve local recovery file, restore access, compare serials, then migrate	State lineage and managed resources verified

Hands-on Lab 6: Migrate Local State and Practice Safe Recovery

Prerequisites

- A disposable Terraform project with one or two local resources.
- A backup directory outside the working repository.

Implementation

1. Apply the project with default local state and record terraform state list.
2. Configure a local backend path under a separate state directory.
3. Run terraform init -migrate-state and confirm the migration.

4. Create a protected state backup with terraform state pull.
5. Run terraform state show for one resource and document the address-to-object binding.
6. Simulate a recovery by copying the project to a new directory, initializing it against the same backend path, and producing a no-change plan without concurrent execution.

Validation Evidence

- The migrated backend contains the authoritative state.
- The original location no longer acts as an independent state source.
- The recovered working directory produces a no-change plan.
- Backup file is outside Git and deleted after validation.

Cleanup

- Destroy managed resources from one working directory only.
- Delete temporary state backups securely.
- Remove the disposable backend directory after confirming no resources remain.

Interview Questions and Model Answers

1. Why does Terraform require state?

It needs a durable mapping between configuration addresses and real objects, plus metadata and dependency information required for efficient lifecycle operations.

2. Does remote state guarantee locking?

No. Locking depends on the selected backend or remote execution platform. Verify backend-specific behavior.

3. When is force-unlock acceptable?

Only after proving that the original writer is no longer active, identifying the exact lock ID, backing up state, and following the recovery procedure.

4. Why not edit terraform.tfstate manually?

It is easy to break lineage, addresses, types, serials, or resource metadata. Use supported commands and backups; direct edits are a last-resort expert operation.

Chapter Summary

- State is critical operational data and may contain sensitive values.
- Remote storage, locking, backups, access control, and recovery are separate required controls.
- State commands can change ownership without changing remote resources.
- Recovery procedures must be tested before an incident.

Chapter 7: Resource Addressing, Meta-Arguments, Iteration, and Lifecycle

Terraform can create many similar resources through `count`, `for_each`, and `dynamic` blocks. The main engineering risk is address stability: changing indexes or keys can cause unwanted replacements even when the underlying business objects are unchanged.

Learning Objectives

- Interpret resource and module addresses.
- Choose correctly between `count` and `for_each`.
- Use `lifecycle`, `depends_on`, `provider`, and other meta-arguments safely.
- Design stable keys and controlled replacement behavior.

Core Concepts

Addresses: A resource instance address identifies an object in configuration and state, such as `local_file.service["api"]`. Address changes are lifecycle events.

count: Creates indexed instances. It is suitable for truly interchangeable items, but deleting an item from the middle of an ordered list can shift indexes.

for_each: Creates instances keyed by a map or set of strings. Stable semantic keys usually provide safer long-term identity.

dynamic blocks: Generate repeated nested blocks when a provider schema requires them. They should not be used when a normal expression or separate resource is clearer.

lifecycle: `create_before_destroy`, `prevent_destroy`, `ignore_changes`, and `replace_triggered_by` alter lifecycle behavior. Each can hide or move risk and must be justified.

Preconditions and postconditions: Lifecycle conditions can enforce assumptions and validate resource results. They improve failure messages but do not replace external functional tests.

Mechanism	Best Fit	Common Risk
<code>count</code>	N identical instances controlled by a number	Index shifts and address churn
<code>for_each</code> map	Named instances with stable keys	Key rename causes destroy/create unless refactored
<code>dynamic</code> block	Repeated provider nested block	Unreadable abstraction
<code>prevent_destroy</code>	High-consequence resource safety net	False confidence; config can be removed
<code>ignore_changes</code>	Shared ownership of selected attribute	Permanent unmanaged drift
<code>create_before_destroy</code>	Replacement with overlap	Quota, naming, duplicate or cost constraints

Reference Configuration or Command Pattern

```
variable "services" {
  type = map(object({
    port = number
    owner = string
  }))
}

resource "local_file" "service" {
  for_each = var.services

  filename = "${path.module}/generated/${each.key}.conf"
  content = jsonencode(each.value)

  lifecycle {
    precondition {
      condition = each.value.port >= 1024 && each.value.port <= 65535
      error_message = "Service ports must be between 1024 and 65535."
    }
  }
}
```

Production Checklist

- `for_each` keys are stable technical identifiers, not mutable display names.
- Lifecycle settings have a documented owner, reason, and review date.
- `ignore_changes` is limited to attributes intentionally owned elsewhere.
- `prevent_destroy` is combined with backups, access control, and recovery procedures.
- Plans are reviewed specifically for address and replacement changes.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Removing one item replaces many resources	count indexes shifted	Migrate to stable <code>for_each</code> keys using moved blocks or state moves	Plan shows only intended removal
Key rename destroys and creates object	<code>for_each</code> address changed	Use moved block and verify provider update behavior	Plan preserves object identity where supported
<code>prevent_destroy</code> did not protect object	Resource block removed from configuration	Use policy, access control, backups, and review in addition	Destructive path blocked by workflow
<code>ignore_changes</code> hides important drift	Attribute excluded too broadly	Narrow or remove ignore rule; assign clear owner	Plan detects material drift

Hands-on Lab 7: Model Stable Service Instances with `for_each`

Prerequisites

- Terraform repository.
- A map of three fictional services.

Implementation

1. Define the services variable and local_file.service resource using for_each.
2. Provide three stable keys such as api, worker, and scheduler.
3. Plan and apply; inspect terraform state list.
4. Change only the owner value for worker and confirm an in-place update.
5. Add a fourth service and confirm one create action.
6. Rename a key, observe the destroy/create plan, cancel it, then restore the original key and document the refactoring requirement.

Validation Evidence

- State addresses use semantic keys.
- Value changes update only the targeted instance.
- Adding an item creates only one new instance.
- The key-rename risk is captured in the runbook.

Cleanup

- Destroy all generated service files through Terraform.
- Delete saved plans.
- Retain the variable schema and lessons in the repository.

Interview Questions and Model Answers

1. When should count be used?

Use it when instances are truly interchangeable and index identity is acceptable, or when a simple conditional zero-or-one resource is clear.

2. Why is for_each often safer?

Stable semantic keys preserve identity when unrelated items are added or removed, reducing index-shift replacement risk.

3. Does prevent_destroy guarantee data safety?

No. It blocks selected Terraform plans while the resource remains in configuration, but cannot replace backups, access controls, provider safeguards, or policy.

4. When is ignore_changes appropriate?

Only when another authorized system owns a specific attribute and the shared-ownership boundary is documented and monitored.

Chapter Summary

- Resource addresses are durable identities within state.
- Stable keys are a major production design decision.
- Meta-arguments change lifecycle semantics and require explicit risk review.
- Conditions improve guardrails but do not replace external acceptance tests.

Chapter 8: Terraform Modules, Composition, Versioning, and Interfaces

Modules package infrastructure patterns for reuse. Effective modules standardize safe decisions and reduce duplication; poor modules create hidden coupling, oversized interfaces, upgrade risk, and layers of indirection.

Learning Objectives

- Explain root and child modules.
- Design clear module inputs, outputs, and composition boundaries.
- Use local, registry, and VCS module sources with version controls.
- Test and document a reusable module.

Core Concepts

Root and child modules: Every working directory is a root module. A module block calls a child module and incorporates its resources into the same state and graph.

Abstraction boundary: A module should represent a coherent capability such as a service configuration bundle, network segment, or application runtime, not merely wrap one resource without adding value.

Interface: Typed inputs, validation, useful outputs, examples, and documented behavior create a stable contract. Avoid exposing every provider argument by default.

Composition: Prefer small modules composed by a root module. Keep policy decisions at the appropriate level and avoid deep module nesting.

Source and version: Registry modules support explicit versions. VCS sources should use immutable tags or commit references. Local modules are useful within a monorepo.

Provider handling: Reusable child modules declare provider requirements but normally receive provider configurations from the caller rather than embedding environment-specific credentials.

Design Question	Preferred Practice	Warning Sign
What capability?	One coherent lifecycle and owner	Module is only a renamed resource
Which inputs?	Small typed contract with safe validation	Dozens of passthrough variables
Which outputs?	Consumer-required stable values	Entire resource objects exposed casually
How versioned?	Immutable semantic releases	Floating branch or unreviewed latest
How composed?	Shallow modules with explicit wiring	Deep hidden module chain

Reference Configuration or Command Pattern

```
# modules/service_file/variables.tf
variable "name" {
  type = string
  validation {
    condition = can(regex("[a-z][a-z0-9-]{2,30}$", var.name))
    error_message = "name must be a lowercase service identifier."
  }
}

variable "settings" {
  type = map(string)
}

# modules/service_file/main.tf
resource "local_file" "this" {
  filename = "${path.module}/../generated/${var.name}.json"
  content = jsonencode(var.settings)
}

# root module
module "api" {
  source = "../modules/service_file"
  name = "api-service"
  settings = { owner = "platform", tier = "critical" }
}
```

Production Checklist

- The module has a clear purpose and owner.
- Inputs are typed, validated, documented, and minimal.
- Outputs expose stable consumer needs without leaking secrets.
- Module sources use reviewed immutable versions.
- README includes example, assumptions, upgrade guidance, validation, and cleanup.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Module upgrade replaces resources	Address, default, or provider behavior changed	Compare versions and plan; use moved blocks and migration notes	Upgrade plan contains approved actions only
Child module cannot use provider alias	Provider mapping not passed from caller	Declare configuration_aliases and pass providers map	Module uses intended provider configuration
Module is difficult to reuse	Environment assumptions embedded inside	Move environment wiring to root module; simplify interface	Two callers use module without forks
Circular dependency between modules	Outputs wired in both directions	Redesign ownership and shared data boundary	Graph becomes acyclic and plan evaluates

Hands-on Lab 8: Build and Consume a Reusable Local Module

Prerequisites

- Repository from earlier chapters.
- A modules directory.

Implementation

1. Create modules/service_file with main.tf, variables.tf, outputs.tf, and README.md.
2. Define validated name and settings inputs.
3. Expose filename and content hash outputs.
4. Call the module twice from the root module with different stable names.
5. Run fmt, validate, plan, apply, and a no-change plan.
6. Change the module implementation in a backward-compatible way and confirm both callers update as intended.

Validation Evidence

- Both module instances appear with clear module addresses.
- Invalid names fail validation.
- Outputs expose only filename and hash.
- README explains usage, contract, and lifecycle.

Cleanup

- Destroy both module instances.
- Delete generated files and plan artifacts.
- Tag the repository exercise version if using Git.

Interview Questions and Model Answers

1. What is the root module?

The configuration in the current working directory used for the Terraform run. Child modules are called from it.

2. Should every resource be wrapped in a module?

No. A module should add a meaningful capability, policy, interface, or reuse boundary. Excessive wrappers increase complexity.

3. How should a reusable module receive provider configuration?

It declares provider requirements, while the caller supplies the appropriate provider configuration, including aliases when required.

4. Why pin module versions?

Immutable versions make upgrades explicit, reviewable, and reproducible and allow migration guidance to match a known release.

Chapter Summary

- Modules encode reusable capabilities and policy, not merely syntax reduction.
- A stable interface is more important than exposing every provider option.
- Composition should remain shallow and ownership explicit.
- Module releases require versioning, documentation, and upgrade testing.

Chapter 9: Importing Existing Infrastructure and Refactoring Safely

Real environments rarely begin as greenfield Terraform projects. Teams must adopt existing objects, rename addresses, split configurations, and retire resources without accidental recreation or deletion.

Learning Objectives

- Explain configuration-driven import and CLI import.
- Use moved blocks and state moves for address refactoring.
- Distinguish forgetting an object from destroying it.
- Plan a controlled brownfield adoption workflow.

Core Concepts

Import contract: Import binds an existing remote object to a Terraform resource address. Configuration must still describe the object; import alone does not create a complete maintainable design.

Configuration-driven import: import blocks make adoption reviewable in normal plans and can support repeated patterns. Provider support determines available identifiers and generated configuration behavior.

Moved blocks: A moved block records an address transition so Terraform can preserve object identity during refactoring rather than plan destroy/create.

State move: terraform state mv performs an immediate state address change. Use backups, exact addresses, peer review, and a follow-up plan.

Forget versus destroy: terraform state rm or a removed block can stop Terraform managing an object without deleting it. This creates an intentionally unmanaged object that requires a new owner.

Brownfield discovery: Inventory, ownership, dependencies, policy, naming, credentials, and lifecycle intent must be established before importing at scale.

Operation	Remote Object	State Binding	Primary Risk
import	Unchanged	Added	Configuration mismatch
moved block	Unchanged	Address changed declaratively	Incorrect source/destination
state mv	Unchanged	Address changed immediately	Wrong state or address
state rm / forget	Unchanged	Removed	Orphaned unmanaged object
destroy	Deleted	Removed	Data loss or outage

Reference Configuration or Command Pattern

```
# Adopt an existing local file into Terraform management
resource "local_file" "legacy" {
  filename = "${path.module}/legacy/legacy.txt"
  content  = "legacy configuration"
}

import {
  to = local_file.legacy
  id = "${path.module}/legacy/legacy.txt"
}

# Preserve identity during a rename
moved {
  from = local_file.legacy
  to   = local_file.adopted
}
```

Production Checklist

- Every imported object has a confirmed owner and lifecycle intent.
- Configuration is written and reviewed before binding at scale.
- A backup and no-change expectation are defined before refactoring state.
- Moved blocks remain long enough for all supported upgrade paths.
- Forgotten objects receive a documented new management owner.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Import succeeds but next plan changes many attributes	Configuration does not match remote defaults or provider normalization	Inspect state and provider docs; model only intended settings	Plan is empty or contains approved convergence
Rename proposes destroy/create	Address changed without moved mapping	Add moved block or execute reviewed state mv	Plan preserves object identity
Imported wrong object	Incorrect ID or target address	Back up state, remove wrong binding, verify remote object, re-import	State address maps to intended ID
state rm caused duplicate creation plan	Configuration still declares resource	Remove or change configuration, or re-import binding	Plan reflects intentional ownership

Hands-on Lab 9: Adopt and Refactor a Brownfield Local Object

Prerequisites

- A disposable legacy directory.
- Terraform project with state backup location.

Implementation

1. Create legacy/legacy.txt manually with known content.
2. Write a matching local_file resource and configuration-driven import block.
3. Run plan, review the import action, and apply.
4. Confirm a no-change plan after adoption.
5. Rename the resource address in configuration and add a moved block.
6. Plan and apply the refactor, then verify the same file remains and no delete/create occurred.

Validation Evidence

- The existing file is adopted without replacement.
- Post-import plan is empty or contains only documented normalization.
- The moved block changes the address while preserving the object.
- State list shows the new address only.

Cleanup

- Destroy the adopted file only if the lab owns it; otherwise forget it deliberately and document ownership.
- Delete state backups securely.
- Retain adoption and refactoring notes in README.

Interview Questions and Model Answers

1. What does terraform import do?

It creates a binding in state between a resource address and an existing remote object. Maintainable configuration is still required.

2. Why use a moved block?

It records a declarative address migration so refactoring can preserve object identity across plans and shared upgrade paths.

3. What is the risk of terraform state rm?

The remote object remains but Terraform forgets it. If configuration still declares the address, Terraform may plan a replacement object.

4. What should precede brownfield import?

Inventory, ownership, dependencies, credentials, lifecycle intent, backup, configuration design, and a no-change acceptance target.

Chapter Summary

- Import adopts existing objects into Terraform state but does not replace architecture work.
- Address refactoring must preserve bindings deliberately.
- Forgetting and destroying are different lifecycle decisions.
- Brownfield adoption requires inventory, ownership, backup, and convergence planning.

Chapter 10: Team Workflow, Remote Operations, and Terraform Foundation Capstone

Terraform becomes an organizational system when multiple engineers and pipelines share modules, state, credentials, approvals, and environments. The capstone integrates the controls from Part A into a reproducible team workflow.

Learning Objectives

- Design a branch-to-plan-to-apply workflow.
- Separate environments and state boundaries deliberately.
- Control credentials, concurrency, approvals, and evidence.
- Complete a foundation capstone with modules, remote-style state, drift handling, and recovery.

Core Concepts

Workflow separation: Pull requests run formatting, validation, linting, security checks, and speculative plans. Protected apply jobs run only from approved commits and authorized identities.

State boundaries: Split state by ownership, lifecycle, blast radius, sensitivity, and change frequency. Do not create a single state for an entire enterprise or one state per trivial resource.

Environment strategy: Separate directories, root modules, variables, or remote workspaces can represent environments. The choice must keep configuration differences explicit and state isolated.

Credentials: CI should use short-lived workload identity or federation. Human apply access should be exceptional, time-bound, and audited.

Concurrency: A team needs locking and run serialization. An approved plan can become stale if another change is applied first.

Evidence: Store sanitized plan summaries, approvals, test results, drift findings, state-backup confirmation, and post-apply validation under retention and access rules.

Pipeline Stage	Purpose	Gate
Format/validate	Reject malformed or inconsistent code	Automated required check
Static/security analysis	Detect policy and misconfiguration risk	No critical unresolved finding
Plan	Produce proposed change from controlled inputs	Plan summary and artifact protection
Approval	Authorize production change	Owner and separation of duties
Apply	Execute exact reviewed plan	Protected identity and serialized run
Verify	Test service and control outcomes	Acceptance checks and rollback decision

Reference Configuration or Command Pattern

```
# Suggested repository layout
terraform-foundation/
modules/
  service_file/
environments/
  dev/
    backend.hcl.example
    main.tf
    variables.tf
    dev.auto.tfvars.example
  prod/
    backend.hcl.example
    main.tf
    variables.tf
.github/workflows/ # or another CI system
.gitignore
README.md
RUNBOOK.md

# CI quality sequence
terraform fmt -check -recursive
terraform init -backend=false
terraform validate
# security/lint tools run here
# controlled backend init and plan run in protected job
```

Production Checklist

- Plans and applies use the same reviewed commit, variables, provider lock file, and state.
- Production apply is protected, serialized, and uses short-lived identity.
- State boundaries match ownership and blast radius.
- Environment differences are explicit and tested.
- Runbooks cover drift, partial apply, lock failure, state restore, and rollback.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Approved plan is stale	Another run changed state after plan creation	Discard plan, update branch, regenerate and reapprove	Applied plan reflects latest serial and commit
Dev change affects prod	Shared state or implicit environment selection	Separate roots/backends and explicit protected inputs	Plan target is unambiguous
CI cannot acquire lock	Concurrent or abandoned run	Identify active run; serialize; recover abandoned lock safely	Single active writer and successful lock lifecycle
Pipeline logs expose secret	Verbose output, plan JSON, or environment leak	Restrict artifacts, redact, rotate, and redesign secret handling	Secret scans and access review pass

Hands-on Lab 10: Terraform Foundation Capstone: Team-Operated Service Configuration Platform

Prerequisites

- Completed Chapters 1-9.
- Git repository and CI system or a documented local simulation.
- No real production credentials or infrastructure.

Implementation

1. Create dev and prod root modules that call the reusable service_file module with different validated inputs and isolated backend paths.
2. Implement formatting, initialization without backend, validation, and plan checks as a CI script or workflow.
3. Create protected backend examples that contain no credentials and document how identity is supplied.
4. Run a dev plan and apply, validate generated service files, and produce a no-change plan.
5. Introduce manual drift, detect it, document the ownership decision, and reconcile through code.
6. Perform a safe resource-address rename with a moved block and prove no replacement.
7. Back up state, recover the project from a clean working directory, and produce a no-change plan.
8. Write RUNBOOK.md covering normal deployment, failed apply, lock recovery, state restore, drift, import, refactor, and cleanup.

Validation Evidence

- Dev and prod have separate state and explicit inputs.
- CI-quality commands are reproducible and fail on invalid configuration.
- The applied result matches the reviewed plan and acceptance tests.
- Drift detection, moved refactor, backup, and recovery evidence are included.
- No state, plan, generated secret, credential, or private artifact is committed.

Cleanup

- Destroy all capstone-managed files and verify both states contain no resources.
- Delete saved plans, plan JSON, temporary state backups, and generated files.
- Retain sanitized configuration, README, RUNBOOK, diagrams, and evidence summary as a portfolio project.

Interview Questions and Model Answers

1. How should Terraform state be divided?

Use ownership, lifecycle, blast radius, sensitivity, dependency, and change frequency. Keep strongly coupled resources together and avoid both monolithic and excessively fragmented state.

2. Why regenerate an approved plan after another apply?

The prior plan was created against an older state and remote observation. Applying it may be rejected or may no longer represent the intended change.

3. What is the preferred CI authentication model?

Short-lived workload identity or federation with least privilege, protected execution context, auditing, and no static credential committed to code or stored broadly.

4. What proves Terraform is production-ready?

Repeatable plans, protected state, controlled dependencies, serialized applies, tested recovery, drift ownership, service-level validation, documented runbooks, and evidence another engineer can reproduce.

Chapter Summary

- Team Terraform workflows combine code review, protected plans, approvals, short-lived identity, locking, and validation.
- State boundaries are architecture decisions.
- A plan is valid only for its exact commit, inputs, provider selections, and state context.
- The capstone demonstrates configuration, lifecycle, state, modules, drift, refactoring, recovery, and operational documentation.

Part A Terraform Command Reference

Run commands only from the intended working directory, backend, workspace, and identity context. Read-only inspection and a current state backup should precede high-risk state operations.

Purpose	Command	Operational Note
Version	terraform version	Record Terraform and provider versions in incident or change evidence.
Format	terraform fmt -check -recursive	Use in local and CI workflows; terraform fmt rewrites files.
Initialize	terraform init	Configures backend and installs modules/providers.
Static initialize	terraform init -backend=false	Useful for validation when backend access is intentionally unavailable.
Validate	terraform validate	Checks configuration consistency, not remote service behavior.
Plan	terraform plan -out=tfplan	Review replacements, destroys, unknowns, and drift.
Inspect saved plan	terraform show tfplan	Use -json for machine analysis; protect artifacts.
Apply reviewed plan	terraform apply tfplan	Applies the exact saved plan.
Refresh-only	terraform plan -refresh-only	Proposes state/output updates without changing remote objects.
State inventory	terraform state list	Confirm selected backend and workspace first.
State detail	terraform state show ADDRESS	May display sensitive attributes; handle output carefully.
Backup state	terraform state pull > backup.tfstate	Store outside source control and delete securely after use.
Move binding	terraform state mv SOURCE DESTINATION	Back up and peer-review exact addresses.
Forget binding	terraform state rm ADDRESS	Remote object remains; assign ownership.
Import	terraform import ADDRESS ID	Configuration-driven import is preferable for reviewable adoption where supported.
Graph	terraform graph	DOT output can reveal infrastructure structure.
Console	terraform console	Useful for type and expression testing.

Purpose	Command	Operational Note
Lock recovery	terraform force-unlock LOCK_ID	Only after proving the original writer is absent.
Destroy plan	terraform plan -destroy	Review before terraform destroy in important environments.

Terraform Foundation Production Readiness Checklist

Repository and Dependencies

- ☐ Required Terraform and provider versions are declared.
- ☐ .terraform.lock.hcl is committed and reviewed.
- ☐ State, plans, credentials, generated secrets, and local working data are ignored.
- ☐ README and runbook identify owners, workflow, validation, and cleanup.

Configuration Quality

- ☐ Variables are typed, described, and validated.
- ☐ Resource keys and addresses are stable.
- ☐ Modules have clear interfaces and immutable versions.
- ☐ Formatting, validation, and lint/security checks run automatically.

State and Backend

- ☐ State storage is encrypted, access-controlled, backed up, and audited.
- ☐ Locking or equivalent serialization is enabled.
- ☐ Restore and force-unlock procedures are tested.
- ☐ State boundaries match ownership and blast radius.

Change Control

- ☐ Plans summarize create, update, replace, and destroy actions.
- ☐ Apply executes the reviewed plan from the reviewed commit.
- ☐ Production identity is short-lived and least-privilege.
- ☐ Functional, security, reliability, and cleanup validation follows apply.

Lifecycle and Recovery

- ☐ Drift has a named owner and response policy.
- ☐ Import and refactoring procedures preserve identity safely.
- ☐ High-consequence replacements have backup, rollback, and approval.
- ☐ Partial apply and state recovery are documented and rehearsed.

Ten Additional Terraform Foundation Projects

1. Multi-Environment File Platform

Create dev, test, and prod roots with isolated state and shared modules; prove no cross-environment action.

- Deliverables: sanitized repository, README, architecture or workflow diagram, plans, validation evidence, cleanup proof, risks, and lessons.

2. Provider Dependency Audit

Build a script that inventories required providers, lock selections, checksums, and upgrade proposals.

- Deliverables: sanitized repository, README, architecture or workflow diagram, plans, validation evidence, cleanup proof, risks, and lessons.

3. State Recovery Drill

Create versioned state backups, simulate working-directory loss, recover, and produce a no-change plan.

- Deliverables: sanitized repository, README, architecture or workflow diagram, plans, validation evidence, cleanup proof, risks, and lessons.

4. Brownfield Adoption Pack

Inventory manually created objects, define import mappings, converge configuration, and publish an adoption runbook.

- Deliverables: sanitized repository, README, architecture or workflow diagram, plans, validation evidence, cleanup proof, risks, and lessons.

5. Address-Stability Demonstration

Compare count and for_each behavior, then refactor using moved blocks with before/after plans.

- Deliverables: sanitized repository, README, architecture or workflow diagram, plans, validation evidence, cleanup proof, risks, and lessons.

6. Module Contract Library

Create three small modules with typed inputs, validation, outputs, examples, tests, and semantic releases.

- Deliverables: sanitized repository, README, architecture or workflow diagram, plans, validation evidence, cleanup proof, risks, and lessons.

7. Drift Management Workflow

Simulate authorized and unauthorized drift and route each through accept, reverse, ignore, or ownership-transfer decisions.

- Deliverables: sanitized repository, README, architecture or workflow diagram, plans, validation evidence, cleanup proof, risks, and lessons.

8. Plan Review Reporter

Parse terraform show -json output and produce a sanitized create/update/replace/destroy summary.

- Deliverables: sanitized repository, README, architecture or workflow diagram, plans, validation evidence, cleanup proof, risks, and lessons.

9. Keyless CI Design

Document a provider-neutral federation model, trust boundary, permissions, approval flow, and incident response.

- Deliverables: sanitized repository, README, architecture or workflow diagram, plans, validation evidence, cleanup proof, risks, and lessons.

10. Terraform Operations Handbook

Create runbooks for init, plan, apply, lock contention, provider failure, partial apply, import, state restore, and destroy.

- Deliverables: sanitized repository, README, architecture or workflow diagram, plans, validation evidence, cleanup proof, risks, and lessons.

Glossary

Term	Definition
Backend	The component that defines Terraform state storage and, depending on type, locking or remote execution behavior.
Child module	A module called by another module through a module block.
Data source	A read-only query that retrieves information from a provider without managing the external object lifecycle.
Dependency lock file	The .terraform.lock.hcl file that records selected provider versions and package checksums.
Drift	A difference between configuration/state expectations and the observed remote object, often caused by out-of-band change.
HCL	HashiCorp Configuration Language, used by Terraform configuration files.
Import	Binding an existing remote object to a Terraform resource address in state.
Meta-argument	Terraform language argument such as count, for_each, depends_on, provider, or lifecycle that changes how a block operates.
Module	A collection of Terraform configuration files managed together as a reusable or root configuration unit.
Plan	Terraform's proposed set of actions for a particular configuration, inputs, provider selection, state, and remote observation.
Provider	A plugin that implements resources and data sources for a remote API or local system.
Resource address	The stable Terraform identifier for a resource or resource instance in configuration and state.
Root module	The Terraform configuration in the current working directory used for a run.
State	Terraform's durable mapping and metadata for managed resource instances.
State locking	Single-writer control used by supported backends during state-changing operations.
Unknown value	A value Terraform cannot determine until a later execution phase.

Term	Definition
Workspace	A named state instance associated with a Terraform configuration and backend; it is not automatically a complete environment-isolation strategy.

Official Terraform Documentation Index

Reference	Official URL
Terraform language overview	https://developer.hashicorp.com/terraform/language
Terraform CLI commands	https://developer.hashicorp.com/terraform/cli/commands
Format and validate configuration	https://developer.hashicorp.com/terraform/cli/code
terraform init	https://developer.hashicorp.com/terraform/cli/commands/init
terraform plan	https://developer.hashicorp.com/terraform/cli/commands/plan
terraform apply	https://developer.hashicorp.com/terraform/cli/commands/apply
Providers	https://developer.hashicorp.com/terraform/language/providers
Provider requirements	https://developer.hashicorp.com/terraform/language/providers/requirements
State overview	https://developer.hashicorp.com/terraform/language/state
Purpose of state	https://developer.hashicorp.com/terraform/language/state/purpose
Remote state	https://developer.hashicorp.com/terraform/language/state/remote
State locking	https://developer.hashicorp.com/terraform/language/state/locking
Backends	https://developer.hashicorp.com/terraform/language/state/backends
Backend configuration	https://developer.hashicorp.com/terraform/language/backend
Modules overview	https://developer.hashicorp.com/terraform/language/modules
Develop modules	https://developer.hashicorp.com/terraform/language/modules/develop
Standard module structure	https://developer.hashicorp.com/terraform/language/modules/develop/structure
Import overview	https://developer.hashicorp.com/terraform/language/import

Reference	Official URL
Import block reference	https://developer.hashicorp.com/terraform/language/block/import
Terraform style guide	https://developer.hashicorp.com/terraform/language/style

Terraform core, providers, backends, hosted products, licensing, regional services, and recommended workflows evolve. Validate current official documentation and provider-specific release notes before production implementation.

Volume 9A Completion

Part A establishes the Terraform foundation required for advanced architecture, policy, testing, multi-environment platforms, HCP Terraform or enterprise operations, and later integration with Ansible.

Next part: Volume 9B - Advanced Terraform Engineering, Testing, Policy, Multi-Environment Architecture, and Enterprise Operations.